

CHAPTER THREE

SYSTEM ANALYSIS AND FEASIBILITY STUDY

3.1 Introduction

This chapter presents the analysis of the proposed **School Learning Management System (LMS)**. System analysis is an important phase in the Software Development Life Cycle (SDLC) because it involves examining the existing educational management processes, identifying their limitations, and determining the functional and non-functional requirements of the proposed system. The analysis provides a clear understanding of the problem domain and serves as the foundation for the system design and implementation presented in the subsequent chapter.

The existing system used by many educational institutions relies heavily on manual processes and fragmented digital solutions for managing academic activities such as student registration, course management, assignment submission, grading, communication, and record keeping. These approaches often result in inefficiencies, duplication of data, delayed dissemination of information, limited accessibility to learning resources, and challenges in monitoring students' academic progress. Such limitations negatively affect the quality of teaching, learning, and administrative decision-making.

The proposed School Learning Management System seeks to address these shortcomings by providing a centralized, web-based platform that integrates the major academic and administrative functions of an educational institution. The system enables administrators to manage users, departments, courses, and academic records; teachers to upload learning materials, create assignments and quizzes, record attendance, and assess students; and students to access course resources, submit assignments, participate in online assessments, and monitor their academic performance through a secure and user-friendly interface.

This chapter evaluates the feasibility of the proposed system from technical, operational, economic, and legal perspectives. It also identifies the functional and non-functional requirements that guide the development of the system. Furthermore, the chapter presents the system modelling techniques used to describe the behaviour and structure of the proposed LMS, including the Context Diagram, Data Flow Diagram (DFD), and Use Case Diagram. These models provide a visual representation of the interactions between users and the system, as well as the movement of data throughout the application.

The outcome of this analysis establishes the foundation for the system architecture, database design, user interface design, and implementation procedures discussed in

Chapter Four. By thoroughly analysing the requirements of the proposed Learning Management System, the study ensures that the final solution effectively addresses the

identified problems while meeting the needs of administrators, teachers, students, and other stakeholders within the educational environment.

3.2 Analysis of the Existing System

The existing academic management system used by many educational institutions is characterised by a combination of manual procedures and isolated digital applications.

While some institutions utilise basic computerised systems for student registration and record keeping, many critical academic processes remain fragmented and inefficient. This lack of integration makes it difficult to manage academic activities effectively and limits timely access to information by administrators, teachers, and students.

The existing system was analysed based on the major functional areas of academic management, as discussed below.

3.2.1 User Management

In the existing system, user accounts for administrators, teachers, and students are often created and managed manually. User information is frequently stored across multiple files or separate systems, making it difficult to maintain consistency and accuracy. The absence of automated role-based access control increases the risk of unauthorised access to sensitive information and complicates user administration.

Weaknesses

- Manual creation and updating of user accounts.
 - Inconsistent user information across different systems.
 - Limited access control mechanisms.
 - Time-consuming user administration.
-

3.2.2 Student Records Management

Student records such as personal information, academic history, enrolment details, attendance records, and examination results are often maintained using paper documents or spreadsheets. This approach makes retrieving, updating, and maintaining records difficult, especially as student numbers increase. Manual record management also increases the possibility of data duplication, loss, and human error.

Weaknesses

- Difficulty retrieving historical records.
 - High risk of data duplication and inconsistencies.
 - Increased possibility of human error.
 - Limited backup and recovery mechanisms.
-

3.2.3 Course Management

Course creation, allocation of teachers, and student enrolment are commonly managed manually or through separate applications. Consequently, administrators spend considerable time updating course information, while students may experience delays in accessing newly assigned courses and learning resources.

Weaknesses

- Manual course allocation.
 - Delayed course updates.
 - Lack of centralised course information.
 - Inefficient monitoring of course enrolment.
-

3.2.4 Assignment Management

Assignment distribution and submission are frequently conducted using printed documents, email, or messaging applications. These methods make it difficult to monitor submission deadlines, organise student submissions, and provide timely feedback.

Teachers also spend additional time sorting and recording submitted assignments manually.

Weaknesses

- Manual assignment submission.
- Difficulty tracking submission deadlines.

- Increased risk of misplaced assignments.
 - Delayed grading and feedback.
-

3.2.5 Attendance Management

Attendance is commonly recorded manually using attendance registers or spreadsheets.

This process is repetitive, susceptible to human error, and requires additional effort when preparing attendance reports. Manual attendance management also makes it difficult to identify absenteeism trends and monitor student participation over time.

Weaknesses

- Paper-based attendance recording.
 - Time-consuming attendance compilation.
 - Difficulty generating attendance reports.
 - Increased likelihood of recording errors.
-

3.2.6 Communication Management

Communication between administrators, teachers, and students is often achieved through notice boards, printed notices, emails, or social media platforms. Since these communication channels are not integrated into a single academic system, important

announcements may not reach all intended recipients promptly, resulting in missed deadlines and reduced collaboration.

Weaknesses

- Fragmented communication channels.
 - Delayed dissemination of information.
 - Poor coordination among stakeholders.
 - Lack of centralised notifications.
-

3.2.7 Report Generation

Generating academic reports such as student performance reports, attendance summaries, course statistics, and administrative reports often requires manual compilation of information from multiple sources. This process is labour-intensive and increases the possibility of inaccuracies in the generated reports.

Weaknesses

- Manual report preparation.
 - Slow report generation.
 - High probability of calculation errors.
 - Limited availability of real-time statistics.
-

3.2.8 Security

The existing system provides limited security mechanisms for protecting academic information. Manual records can easily be damaged or misplaced, while simple digital systems may lack adequate authentication, authorisation, and audit mechanisms. As a result, confidential academic information may be exposed to unauthorised individuals.

Weaknesses

- Weak authentication mechanisms.
 - Limited role-based access control.
 - Poor data protection practices.
 - Insufficient backup and recovery procedures.
 - Lack of audit trails for monitoring system activities.
-

3.2.9 Summary of the Existing System Analysis

The analysis of the existing system reveals several shortcomings that affect the efficient management of academic activities. Manual processes, fragmented information systems, delayed communication, inefficient record management, inadequate security, and limited reporting capabilities reduce institutional productivity and negatively impact teaching and learning. These limitations demonstrate the need for a comprehensive, integrated Learning Management System that centralises academic operations, automates routine

processes, improves communication, strengthens data security, and provides timely access to educational information for all authorised users. The proposed School Learning Management System is designed to address these challenges by providing a secure, web-based platform that enhances the efficiency and effectiveness of educational management.

3.3 Existing System Workflow

The workflow of the existing academic management system is largely manual and involves several disconnected processes performed by different stakeholders. Students submit requests, assignments, and other academic documents physically or through separate communication platforms. Teachers process these documents manually before forwarding relevant information to departmental offices or the academic administration for further action. Records are then stored in filing cabinets, spreadsheets, or independent databases.

This workflow requires multiple manual interventions, resulting in delays, duplication of effort, and increased chances of errors. Information often passes through several departments before reaching its destination, making it difficult to monitor academic activities efficiently and retrieve records promptly.

Figure 3.1: Workflow of the Existing System

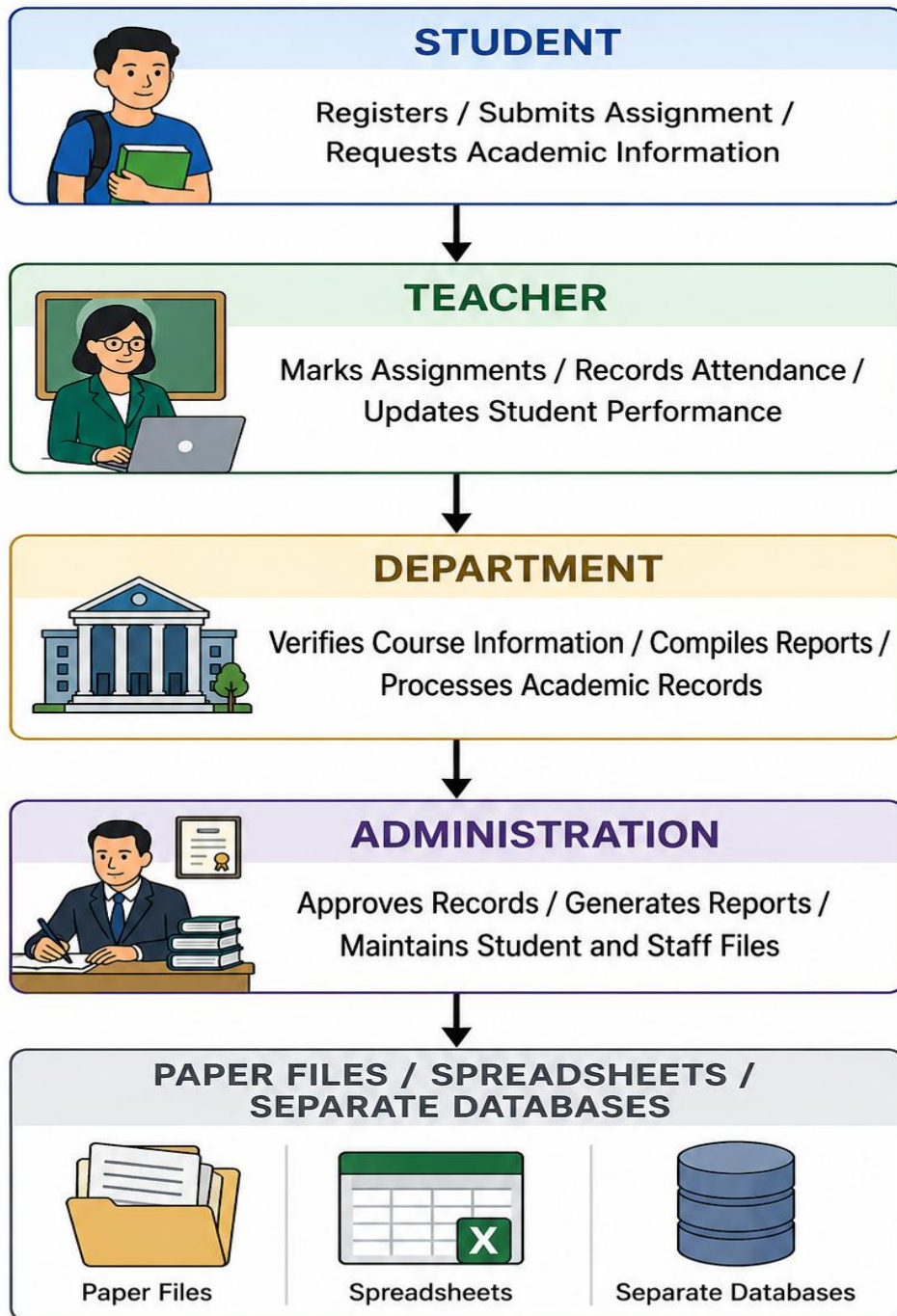


Figure 3.1: Workflow of the Existing System

Explanation

The existing workflow begins with students submitting academic requests, assignments, or registration information to their teachers. Teachers manually process these submissions by recording attendance, assessing assignments, and updating students' academic performance. The processed information is subsequently forwarded to the relevant department, where it is verified and consolidated before being sent to the administration for approval and record keeping.

Since most of these activities depend on manual documentation or isolated software applications, information is repeatedly transferred between offices before it is permanently stored. This approach increases processing time, creates opportunities for data duplication and human error, and makes retrieving academic records difficult.

Furthermore, communication between administrators, teachers, and students is often delayed because there is no centralised platform for sharing academic information.

The identified weaknesses of this workflow justify the development of a fully integrated Learning Management System that automates academic processes, centralises data management, improves communication, and provides authorised users with real-time access to educational information.

3.4 Proposed System Workflow

The proposed School Learning Management System (LMS) introduces a centralized and automated workflow that integrates all major academic and administrative activities into a single web-based platform. Unlike the existing manual system, users interact directly with the system through a secure login interface, where access to features is determined by their assigned roles. The system processes user requests, communicates with the database through the backend API, and provides real-time responses to authorised users. The workflow is designed to reduce manual intervention, improve communication, enhance data security, and ensure that academic information is available whenever required. Administrators, teachers, and students each access customised dashboards containing only the functions relevant to their responsibilities.

Figure 3.2: Workflow of the Proposed Learning Management System

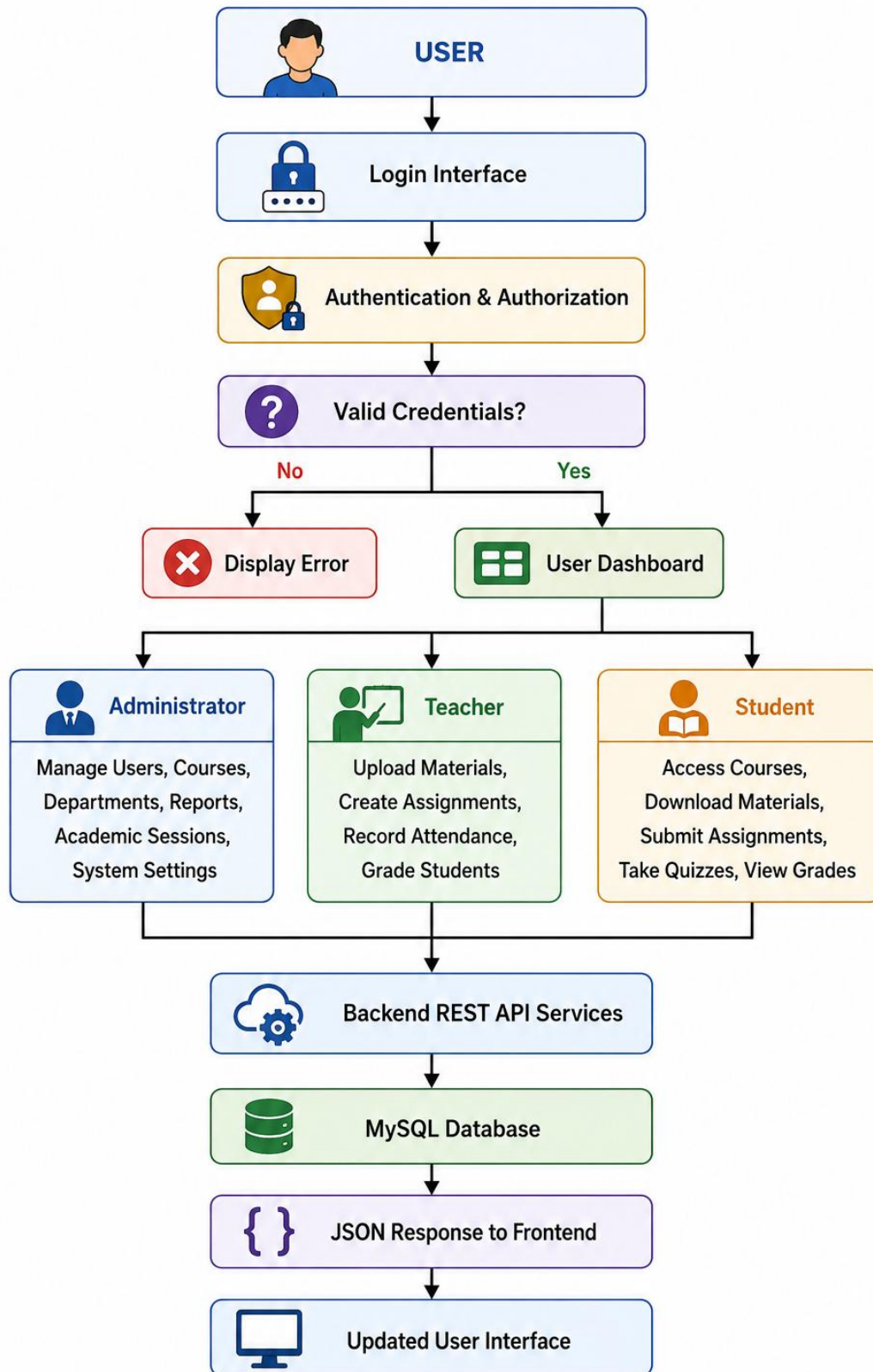


Figure 3.2: Workflow of the Proposed Learning Management System

Explanation of the Proposed Workflow

The workflow begins when a user accesses the system through the login page and enters valid authentication credentials. The backend verifies the credentials, checks the user's role, and establishes a secure session. Once authenticated, the user is redirected to a role-specific dashboard.

Administrators are provided with tools for managing users, departments, academic sessions, courses, reports, and other institutional resources. Teachers can create courses, upload learning materials, create assignments and quizzes, record attendance, assess students, and monitor academic performance. Students can access enrolled courses, download learning materials, submit assignments, participate in online assessments, view grades, and receive announcements.

Every action performed by a user is transmitted to the backend through secure RESTful API endpoints. The backend validates the request, performs the necessary business logic, and interacts with the MySQL database using prepared statements to maintain data integrity and security. The API returns responses in JSON format, which are processed by the frontend to update the user interface dynamically without exposing backend implementation details.

Compared to the existing workflow, the proposed system significantly reduces paperwork, eliminates duplication of records, improves communication among

stakeholders, enhances data security through authentication and role-based access control, and enables faster retrieval of academic information. The automated workflow also supports real-time processing of academic activities, thereby improving the efficiency and effectiveness of educational administration.

3.5 Functional Requirements

Functional requirements describe the specific operations and services that the proposed School Learning Management System (LMS) must provide to satisfy the needs of its users. The system is designed using role-based access control, ensuring that each category of user has access only to the functions relevant to their responsibilities. The major functional requirements are grouped according to the three primary user roles: Administrator, Teacher, and Student.

3.5.1 Functional Requirements for the Administrator

The administrator is responsible for managing the overall operation of the Learning Management System. The system shall enable the administrator to perform the following functions:

- Securely log into and log out of the system.
- Access an administrative dashboard displaying key system statistics.
- Create, update, deactivate, and manage user accounts.

- Assign user roles and permissions.
 - Manage departments and academic programmes.
 - Create, update, and manage courses.
 - Assign teachers to courses.
 - Manage academic years, semesters, and sessions.
 - View student enrolment information.
 - Generate academic and administrative reports.
 - View system audit logs and user activities.
 - Configure system settings.
 - Perform database backup and recovery operations.
 - Manage announcements and institutional notifications.
 - Monitor overall system performance and usage.
-

3.5.2 Functional Requirements for the Teacher

Teachers are responsible for delivering course content and assessing students. The proposed system shall enable teachers to perform the following functions:

- Securely log into and log out of the system.
- Access a personalised teacher dashboard.
- View assigned courses.

- Create and update course information where permitted.
 - Upload, edit, archive, and delete learning materials.
 - Create assignments with submission deadlines.
 - Create and manage online quizzes and examinations.
 - Grade assignments and quizzes.
 - Record and manage student attendance.
 - View enrolled students for each course.
 - Publish announcements to students.
 - View student submissions.
 - Provide feedback on submitted assignments.
 - Monitor student academic performance.
 - Generate course-related reports.
-

3.5.3 Functional Requirements for the Student

Students interact with the system primarily to access learning resources and participate in academic activities. The system shall enable students to perform the following functions:

- Securely log into and log out of the system.
- Access a personalised student dashboard.
- View registered or enrolled courses.

- Access and download learning materials.
 - Submit assignments electronically.
 - Participate in online quizzes and examinations.
 - View assignment deadlines.
 - Receive announcements and notifications.
 - View grades and academic performance.
 - Monitor attendance records.
 - Update permitted profile information.
 - Communicate with instructors through the system where applicable.
-

3.5.4 General Functional Requirements

In addition to user-specific functions, the system shall provide the following general capabilities:

- Authenticate users before granting system access.
- Enforce role-based access control to protect sensitive information.
- Validate user input before processing requests.
- Store academic information securely in a centralised database.
- Support secure file uploads and downloads.
- Record important system activities through audit logs.

- Generate reports in real time.
- Maintain data integrity through database validation and constraints.
- Support RESTful API communication between the frontend and backend.
- Provide consistent JSON responses for all API requests.
- Automatically manage user sessions and log users out after periods of inactivity.

Summary

The functional requirements define the essential services that the proposed School Learning Management System must provide to support effective academic administration and online learning. By separating system functions according to user roles, the proposed solution ensures that administrators, teachers, and students are provided with appropriate tools and permissions while maintaining the security, integrity, and efficiency of the overall system. These requirements serve as the foundation for the system design and implementation presented in the next chapter.

3.6 Non-Functional Requirements

Non-functional requirements describe the quality attributes and operational characteristics that the proposed School Learning Management System (LMS) must possess to ensure efficient, reliable, secure, and user-friendly operation. Unlike functional

requirements, which define what the system should do, non-functional requirements specify how well the system should perform under different conditions.

3.6.1 Performance Requirements

The proposed LMS shall provide efficient and timely processing of user requests to ensure a satisfactory user experience.

The system shall:

- Load web pages within **three (3) seconds** under normal network conditions.
- Process API requests within **two (2) seconds** for standard operations.
- Support at least **500 concurrent users** without significant degradation in performance.
- Allow users to upload learning materials of up to **100 MB** per file.
- Process assignment submissions immediately after upload.
- Generate reports within **five (5) seconds** for standard datasets.
- Minimise server response time through efficient database queries and caching where applicable.

3.6.2 Security Requirements

Since the system stores confidential academic information, appropriate security mechanisms shall be implemented to protect user data and system resources.

The system shall:

- Encrypt user passwords using the **bcrypt hashing algorithm**.
- Implement **Role-Based Access Control (RBAC)** to restrict access based on user roles.
- Validate and sanitise all user inputs to prevent malicious attacks.
- Use **prepared SQL statements** to prevent SQL injection.
- Protect against **Cross-Site Scripting (XSS)** attacks through input validation and output encoding.
- Protect against **Cross-Site Request Forgery (CSRF)** attacks using security tokens.
- Automatically terminate inactive user sessions after a predefined timeout period.
- Record login attempts and critical user activities in audit logs.
- Support secure communication using **HTTPS** when deployed to a production environment.
- Restrict file uploads to approved file types and sizes.

3.6.3 Reliability Requirements

The system shall operate consistently and maintain data integrity during normal operation and unexpected failures.

The system shall:

- Maintain a minimum availability of **99% uptime** during normal operation.
 - Automatically back up the database at scheduled intervals.
 - Maintain transaction integrity during database operations.
 - Recover gracefully from unexpected system failures.
 - Log application errors for troubleshooting and maintenance.
 - Prevent data corruption through proper database constraints and validation.
 - Ensure uploaded files remain accessible unless intentionally removed by authorised users.
-

3.6.4 Usability Requirements

The system shall provide an intuitive and user-friendly interface that supports users with varying levels of computer literacy.

The system shall:

- Provide a responsive interface that functions correctly on desktops, tablets, and smartphones.
- Maintain consistent navigation throughout all system pages.
- Display clear labels, icons, and instructions for all major functions.
- Present informative validation and error messages.

- Minimise the number of steps required to complete common tasks.
 - Support accessibility features such as readable fonts, sufficient colour contrast, and keyboard navigation where applicable.
 - Provide dashboards customised to the responsibilities of administrators, teachers, and students.
-

3.6.5 Maintainability Requirements

The proposed LMS shall be designed to simplify future maintenance, upgrades, and feature enhancements.

The system shall:

- Follow a modular software architecture.
 - Separate the frontend from the backend through RESTful APIs.
 - Use reusable components to reduce code duplication.
 - Maintain clear documentation for APIs, database structures, and source code.
 - Record application logs to facilitate debugging.
 - Support future integration with mobile applications and third-party educational services.
-

3.6.6 Scalability Requirements

The system shall support future institutional growth without requiring significant architectural changes.

The system shall:

- Allow additional users, departments, and courses to be added without affecting system performance.
 - Support expansion to multiple campuses or educational institutions.
 - Accommodate increasing numbers of uploaded learning materials and assignments.
 - Allow additional functional modules, such as online payments, virtual classrooms, and library management, to be integrated in future versions.
-

3.6.7 Compatibility Requirements

The LMS shall operate consistently across commonly used hardware and software platforms.

The system shall:

- Support modern web browsers including **Google Chrome, Microsoft Edge, Mozilla Firefox, and Safari.**
- Operate on Windows, Linux, macOS, and Android devices through a web browser.

- Integrate with MySQL database servers.
 - Support RESTful API communication using JSON data exchange.
-

3.6.8 Availability Requirements

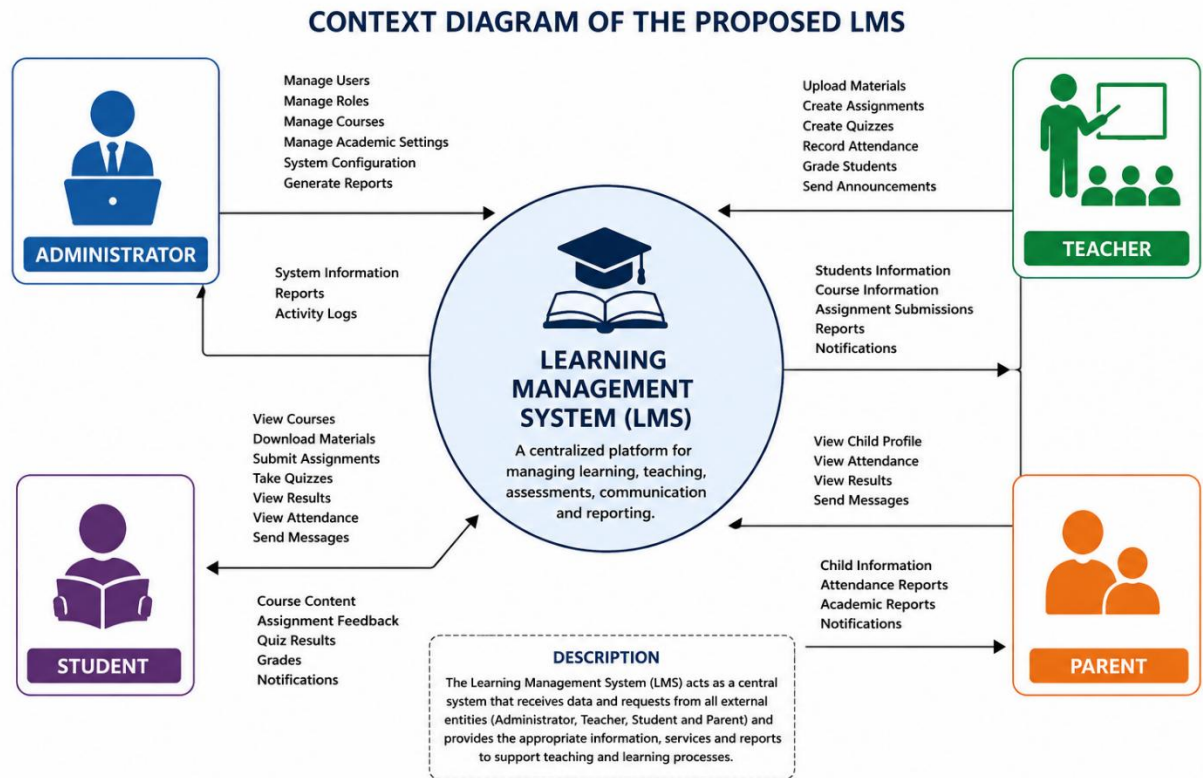
The system shall be accessible whenever authorised users require its services.

The system shall:

- Be available **24 hours a day and 7 days a week**, except during scheduled maintenance.
 - Notify users in advance of planned maintenance activities.
 - Resume normal operation promptly after maintenance or unexpected outages.
-

3.6.9 Context Diagram

Figure 3.3: Context Diagram of the Proposed LMS

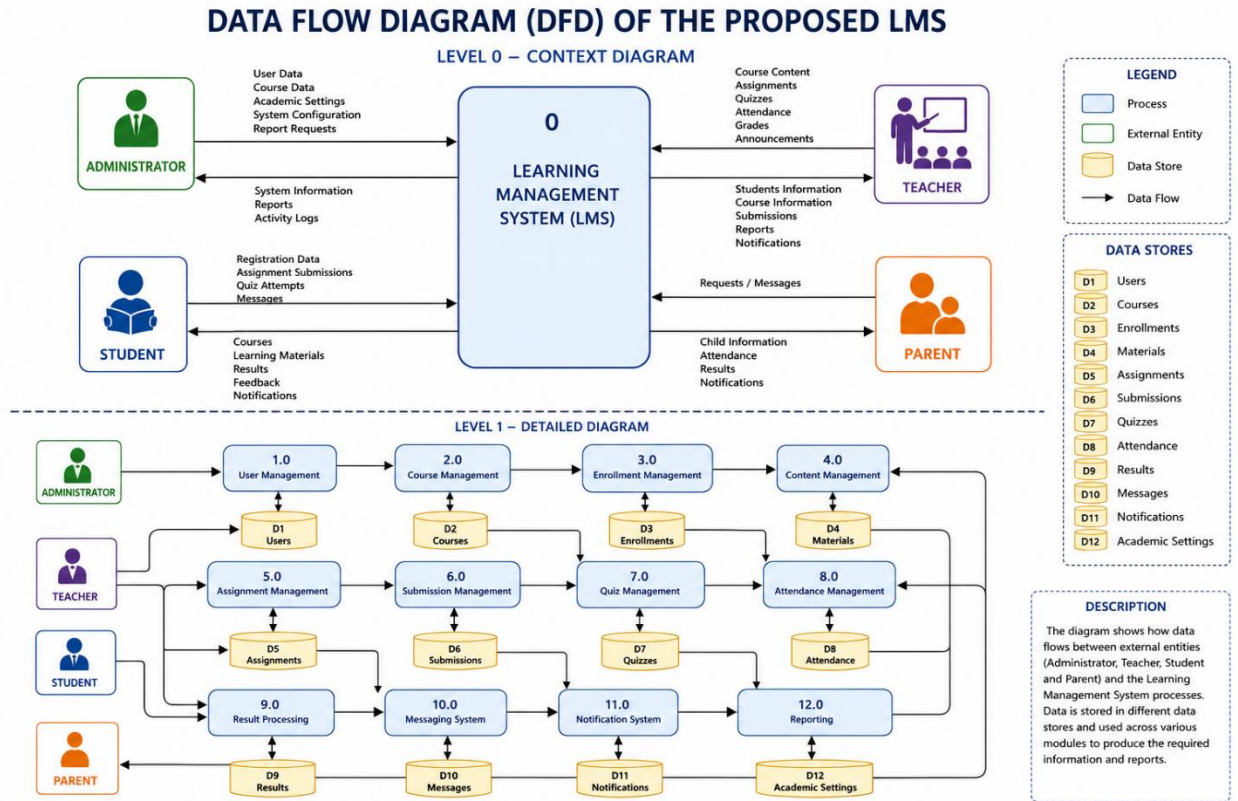


Explanation

The Context Diagram illustrates the interaction between external entities and the Learning Management System. Administrators manage users, courses, and reports. Teachers upload learning materials, create assignments, and record grades. Students access learning resources, submit assignments, and view academic results. The LMS acts as the central processing system that facilitates these interactions.

3.6.10 Data Flow Diagram (Level 0)

Figure 3.4: Data Flow Diagram (DFD Level 0)



Explanation

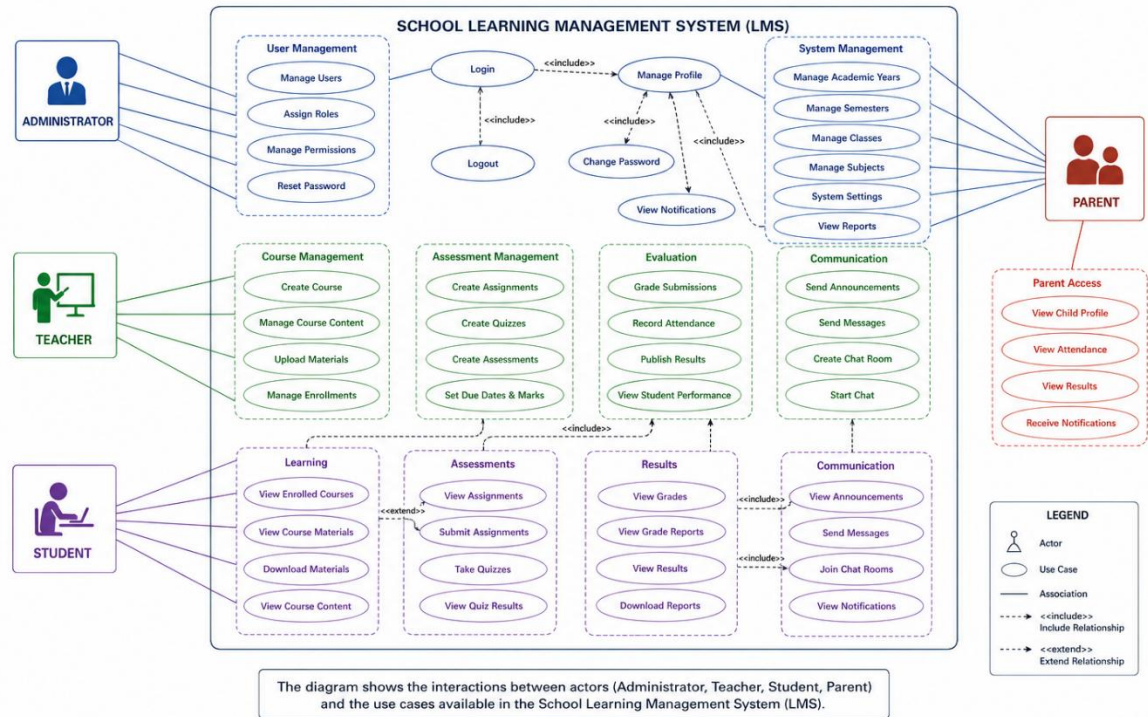
The Data Flow Diagram illustrates how information flows within the proposed system.

Administrators manage users and courses, teachers upload learning materials and assessments, and students access educational resources and submit assignments. All transactions are stored and retrieved from the LMS database.

3.6.11 Use Case Diagram

Figure 3.5: Use Case Diagram of the Proposed LMS

USE CASE DIAGRAM SCHOOL LEARNING MANAGEMENT SYSTEM (LMS)



Explanation

The Use Case Diagram identifies the major interactions between users and the system.

Administrators perform management and reporting functions, teachers manage academic activities, and students access learning resources and monitor their academic progress.

Summary

The non-functional requirements establish the quality standards that the proposed School Learning Management System must satisfy to operate effectively. These requirements address critical aspects such as performance, security, reliability, usability,

maintainability, scalability, compatibility, and availability. Collectively, they ensure that the system not only performs its intended functions but also delivers a secure, efficient, reliable, and user-friendly platform capable of supporting the academic and administrative activities of the institution while accommodating future growth and technological advancements.

3.7 System Requirements

The successful implementation of the proposed School Learning Management System (LMS) depends on the availability of appropriate hardware and software resources. These requirements ensure that the system operates efficiently, securely, and reliably while supporting multiple users simultaneously. The requirements are categorised into hardware requirements and software requirements.

3.7.1 Hardware Requirements

The hardware requirements specify the minimum and recommended computing resources needed to host and access the proposed LMS.

Server Hardware Requirements

The server hosts the backend application, RESTful APIs, database, and uploaded learning resources. To ensure efficient system performance, the following minimum hardware specifications are recommended.

Component	Minimum Requirement	Recommended Requirement
Processor	Intel Core i5 (8th Generation) or AMD Ryzen 5	Intel Core i7 or AMD Ryzen 7
Memory (RAM)	8 GB	16 GB or higher
Storage	100 GB SSD	500 GB SSD
Network	Broadband Internet Connection	High-speed Fibre Internet
Backup Storage	External Hard Drive	Cloud Backup Service
Power Supply	UPS	UPS with Generator Backup

Client Hardware Requirements

Client computers are used by administrators, teachers, and students to access the LMS through a web browser.

Component	Minimum Requirement
Processor	Dual-Core Processor (2.0 GHz or higher)
Memory (RAM)	4 GB
Storage	20 GB Free Disk Space
Display	1366 × 768 Resolution or Higher
Internet Connection	Stable Broadband Connection
Input Devices	Keyboard and Mouse or Touchscreen

3.7.2 Software Requirements

The software requirements describe the applications, operating systems, databases, and development tools required for developing, deploying, and operating the proposed system.

Server Software Requirements

Software	Version / Requirement
Operating System	Windows 10/11, Ubuntu Linux, or Windows Server
Web Server	Apache HTTP Server
Programming Language	PHP 8.x

Software	Version / Requirement
Database Management System	MySQL 8.x
API Format	RESTful API using JSON
Local Development Environment	XAMPP
Database Administration	phpMyAdmin

Client Software Requirements

Users access the LMS using a standard web browser without installing additional software.

Software	Requirement
Web Browser	Google Chrome (latest version)
Web Browser	Microsoft Edge (latest version)
Web Browser	Mozilla Firefox (latest version)
Web Browser	Safari (latest version for macOS users)
Internet Connection	Stable Internet Access

Development Software Requirements

The system was developed using modern software engineering tools that support version control, API development, debugging, and collaborative development.

Software	Purpose
Visual Studio Code	Source code development
Git	Version control
GitHub	Source code repository and collaboration
Postman	API testing and documentation
XAMPP	Local web server and database environment
phpMyAdmin	Database administration
Composer	PHP dependency management
Google Chrome Developer Tools	Frontend debugging and API inspection

3.7.3 Network Requirements

The Learning Management System is designed as a web-based application; therefore, reliable network connectivity is essential for both server and client operations.

The system requires:

- A stable broadband Internet connection.
- Secure communication using the HTTP/HTTPS protocol.

- Local Area Network (LAN) or Wireless LAN (WLAN) support within the institution.
 - Continuous network availability to ensure uninterrupted access to learning resources and academic services.
-

3.7.4 Browser Compatibility

To ensure accessibility across different computing platforms, the proposed LMS is compatible with modern web browsers, including:

- Google Chrome
- Microsoft Edge
- Mozilla Firefox
- Safari
- Opera

The user interface is responsive and adapts automatically to desktops, laptops, tablets, and smartphones, providing a consistent user experience across different screen sizes.

3.7.5 Summary

The proposed School Learning Management System requires modest hardware resources and widely available software technologies, making it suitable for deployment in

educational institutions of varying sizes. The use of PHP, MySQL, Apache, RESTful APIs, and modern web browsers ensures compatibility, portability, and ease of maintenance. Furthermore, the adoption of development tools such as Visual Studio Code, Git, GitHub, and Postman supports efficient software development, testing, and future system enhancement. These requirements provide the technical foundation necessary for the successful implementation and long-term operation of the proposed Learning Management System.

3.8 Database Requirements

The database is one of the most important components of the proposed School Learning Management System (LMS). It serves as the central repository for storing, organising, and managing all academic and administrative information generated by the system. The database maintains records relating to users, courses, departments, enrolments, learning materials, assignments, grades, attendance, announcements, and other institutional data. A well-designed database ensures data consistency, efficient retrieval of information, and secure management of academic records.

3.8.1 Database Management System Selection

The proposed Learning Management System uses **MySQL** as its Database Management System (DBMS). MySQL was selected because it is an open-source, reliable, and high-performance relational database management system that integrates effectively with PHP and supports web-based application development.

The selection of MySQL was based on the following advantages:

- It is open-source and freely available, making it cost-effective for educational institutions.
- It provides high-speed data processing and efficient query execution.
- It integrates seamlessly with PHP and Apache through the XAMPP development environment.
- It supports relational database structures suitable for complex academic information.
- It offers strong security features such as user authentication and access privileges.
- It supports backup, recovery, and transaction management.
- It is widely adopted in educational and enterprise web applications, ensuring long-term community support and documentation.

3.8.2 Database Normalisation

Database normalisation is the process of organising data into related tables to minimise redundancy and improve data integrity. The database for the proposed LMS was designed following the principles of relational database normalisation.

The database satisfies the following normal forms:

First Normal Form (1NF)

The database ensures that:

- Each table contains atomic (indivisible) values.
- Every record is uniquely identified by a primary key.
- Repeating groups of data have been eliminated.

Second Normal Form (2NF)

The database satisfies Second Normal Form by ensuring that:

- All non-key attributes depend entirely on the primary key.
- Partial dependencies have been removed.
- Related information is stored in separate tables.

Third Normal Form (3NF)

The database satisfies Third Normal Form by ensuring that:

- There are no transitive dependencies.
- Every non-key attribute depends only on the primary key.
- Data duplication is minimised.

- Data consistency is maintained throughout the database.

The application of these normalisation principles improves storage efficiency, simplifies maintenance, and reduces the possibility of update anomalies.

3.8.3 Database Relationships

The proposed database follows a relational model in which related tables are connected using **primary keys** and **foreign keys**. These relationships maintain referential integrity and ensure consistency among the various academic records.

Some of the major relationships in the database include:

Parent Table	Child Table	Relationship
Roles	Users	One-to-Many
Departments	Courses	One-to-Many
Users (Teacher)	Courses	One-to-Many
Courses	Materials	One-to-Many
Courses	Assignments	One-to-Many
Courses	Quizzes	One-to-Many
Courses	Attendance	One-to-Many

Parent Table	Child Table	Relationship
Students	Enrolments	One-to-Many
Courses	Enrolments	One-to-Many
Assignments	Submissions	One-to-Many
Students	Submissions	One-to-Many
Students	Grades	One-to-Many
Courses	Grades	One-to-Many
Users	Audit Logs	One-to-Many

These relationships enable the system to retrieve related academic information efficiently while maintaining data accuracy and consistency.

3.8.4 Database Security

Since the database stores confidential institutional information, several security mechanisms have been incorporated to protect data from unauthorised access and malicious activities.

The proposed system implements the following database security measures:

- User authentication before accessing protected resources.

- Role-Based Access Control (RBAC) to restrict database operations according to user responsibilities.
- Password encryption using the **bcrypt** hashing algorithm.
- Prepared SQL statements to prevent SQL injection attacks.
- Input validation and sanitisation before database operations.
- Foreign key constraints to preserve referential integrity.
- Database transaction management to maintain consistency during updates.
- Automatic recording of important activities in audit logs.
- Scheduled database backups to minimise data loss.
- Recovery procedures for restoring data in the event of system failure.
- Restricted administrative privileges to authorised personnel only.

These measures help ensure the confidentiality, integrity, and availability of academic information stored within the database.

3.8.5 Database Entities

The proposed Learning Management System database consists of several interconnected entities that support the various academic and administrative functions of the system. The major entities include:

- Users

- Roles
- Departments
- Courses
- Student Enrolments
- Learning Materials
- Assignments
- Assignment Submissions
- Quizzes
- Quiz Attempts
- Grades
- Attendance Records
- Announcements
- Academic Sessions
- Audit Logs

Each entity represents a specific aspect of the educational process and is linked to other entities through well-defined relationships.

3.8.6 Database Design Considerations

The database was designed with the following objectives:

- Eliminate unnecessary data redundancy.
 - Ensure high data integrity through relational constraints.
 - Support efficient searching, sorting, and reporting.
 - Provide scalability for future institutional growth.
 - Facilitate secure storage of academic records.
 - Support RESTful API operations through efficient query execution.
 - Enable easy maintenance and future database expansion.
-

Summary

The proposed School Learning Management System employs **MySQL**, a robust relational database management system, to provide efficient and secure storage of academic and administrative information. The database has been normalised up to the **Third Normal Form (3NF)** to minimise redundancy and improve consistency.

Relationships among entities are maintained using primary and foreign keys, while comprehensive security mechanisms—including authentication, role-based access control, prepared statements, password hashing, and scheduled backups—protect sensitive institutional data. Consequently, the database provides a reliable and scalable foundation for supporting the functional requirements of the proposed Learning Management System.

3.9 API Architecture

The proposed School Learning Management System (LMS) adopts an **API-first architecture**, in which the frontend and backend are developed as separate components that communicate through **Representational State Transfer (REST)** APIs. This architectural approach promotes modularity, scalability, maintainability, and reusability by ensuring that the user interface is independent of the application logic and database operations.

In the proposed architecture, users interact with the system through a web-based frontend developed using HTML5, CSS3, and JavaScript. The frontend sends requests to the backend through RESTful API endpoints using standard HTTP methods. The backend, developed with PHP, processes these requests, performs authentication and business logic, communicates with the MySQL database, and returns responses in JavaScript Object Notation (JSON) format.

This separation of concerns enables the backend APIs to serve multiple client applications, including web, mobile, and desktop applications, without modifying the underlying business logic.

Figure 3.6: API Architecture of the Proposed Learning Management System

User



Frontend (HTML, CSS, JavaScript)



HTTP Request (Fetch API/AJAX)



RESTful API Endpoints (PHP)



Authentication & Authorization



Business Logic Layer



MySQL Database Server



Database Response





JSON Response (REST API)



Frontend Updates User Interface

3.9.1 RESTful API Design

The proposed LMS follows the principles of **Representational State Transfer (REST)**.

REST is an architectural style for designing web services that enables communication between distributed software components using standard HTTP protocols.

Each resource within the system, such as users, courses, assignments, learning materials, attendance records, and grades, is represented by a unique API endpoint. These endpoints perform specific operations without exposing the internal implementation of the application.

Examples of API resources include:

- /api/auth/login
- /api/users
- /api/courses
- /api/materials

- /api/assignments
- /api/quizzes
- /api/grades
- /api/attendance

This resource-oriented approach simplifies integration, maintenance, and future system expansion.

3.9.2 HTTP Methods

The RESTful API uses standard HTTP methods to perform different operations on system resources.

HTTP Method	Purpose	Example
GET	Retrieve information	Retrieve course materials
POST	Create new resources	Create a course or upload material
PUT	Update existing resources	Update course details or grades
DELETE	Remove resources	Delete a course or learning material

Using standard HTTP methods improves consistency and makes the API easier to understand and integrate with different client applications.

3.9.3 JSON Data Exchange

The proposed system uses **JavaScript Object Notation (JSON)** as the standard data exchange format between the frontend and backend.

When a user performs an operation, the frontend sends data to the backend as a JSON request. After processing the request, the backend returns a JSON response containing the requested information or an appropriate error message.

A typical API response includes:

- Operation status
- Response message
- Requested data
- HTTP status code
- Timestamp

JSON was selected because it is lightweight, easy to read, language-independent, and supported by modern web browsers and programming languages.

3.9.4 Stateless Communication

The RESTful APIs are designed using the principle of **stateless communication**. Under this approach, every client request contains all the information required for the server to

process that request. The server does not rely on previous requests to determine the current request's context.

The benefits of stateless communication include:

- Improved scalability.
- Reduced server memory usage.
- Easier load balancing.
- Better fault tolerance.
- Simplified API maintenance.

Although user authentication is managed through secure sessions, each API request is independently validated before processing to ensure that only authorised users can access protected resources.

3.9.5 Authentication and Authorization

Before granting access to protected resources, every API request passes through an authentication and authorisation layer.

The authentication component verifies the user's identity using secure login credentials and establishes a valid user session. Once authenticated, the authorisation component determines whether the user has sufficient privileges to perform the requested operation based on **Role-Based Access Control (RBAC)**.

The system defines three primary user roles:

- Administrator
- Teacher
- Student

Each role is granted access only to the resources and operations relevant to its responsibilities, thereby enhancing system security and protecting confidential academic information.

3.9.6 Advantages of the API-First Architecture

The API-first approach provides several advantages over traditional monolithic web applications in which the frontend and backend are tightly coupled.

The proposed architecture offers the following benefits:

- Clear separation between the frontend and backend.
- Improved maintainability through modular development.
- Easier testing of backend services using tools such as Postman.
- Reusability of APIs across web, mobile, and desktop applications.
- Simplified integration with third-party educational systems.
- Improved scalability through independent deployment of frontend and backend components.

- Faster development by allowing frontend and backend teams to work independently.
 - Consistent communication through standard JSON responses.
-

Summary

The proposed School Learning Management System adopts an **API-first architecture** that separates the presentation layer from the business logic and database layers.

Communication between the frontend and backend is achieved through **RESTful APIs**, **HTTP methods**, and **JSON** data exchange, while **stateless request processing** improves scalability and maintainability. Authentication and role-based authorisation ensure that only authorised users can access protected resources. This architecture provides a secure, flexible, and scalable foundation capable of supporting future enhancements, including mobile applications and integration with external educational platforms.

3.10 Module Description

The proposed School Learning Management System (LMS) is organised into several functional modules, each responsible for performing a specific set of academic or administrative tasks. This modular design improves maintainability, scalability, and system organisation by separating related functionalities into independent components.

Each module communicates with the backend through RESTful APIs and interacts with the MySQL database to perform authorised operations.

3.10.1 Authentication Module

The Authentication Module is responsible for verifying user identities before granting access to the system. It ensures that only authorised users can access protected resources while maintaining secure user sessions.

Functions

- User login
 - User logout
 - Session management
 - Password verification
 - Role-Based Access Control (RBAC)
 - User authentication
 - Session timeout management
 - Password encryption using bcrypt
-

3.10.2 User Management Module

This module enables administrators to manage users within the system. It maintains records for administrators, teachers, and students while controlling user roles and permissions.

Functions

- Create user accounts
 - Update user information
 - Delete or deactivate users
 - Assign user roles
 - Manage departments
 - Reset passwords
 - Search and filter users
 - View user activity
-

3.10.3 Course Management Module

The Course Management Module manages all academic courses offered by the institution. It allows administrators and authorised teachers to create and organise courses while assigning instructors and enrolling students.

Functions

- Create courses

- Edit course information
 - Delete or archive courses
 - Assign teachers to courses
 - Manage course enrolments
 - View course details
 - Search available courses
 - Organise courses by department and academic session
-

3.10.4 Learning Materials Module

This module manages digital learning resources uploaded by teachers. Students can access and download materials for their enrolled courses.

Functions

- Upload learning materials
- Download learning materials
- Archive or restore materials
- Delete learning materials
- Organise materials by course
- Search learning resources
- Preview supported files

- Manage uploaded documents
-

3.10.5 Assignment Management Module

The Assignment Management Module enables teachers to create coursework while allowing students to submit assignments electronically.

Functions

- Create assignments
 - Edit assignment details
 - Set submission deadlines
 - Upload assignment instructions
 - Submit assignments
 - View student submissions
 - Grade assignments
 - Provide feedback
 - Monitor submission status
-

3.10.6 Quiz and Assessment Module

This module supports online quizzes and assessments that evaluate students' understanding of course content.

Functions

- Create quizzes
 - Add objective and subjective questions
 - Configure quiz duration
 - Schedule quizzes
 - Submit quiz attempts
 - Automatically score objective questions
 - Record quiz results
 - Generate assessment reports
-

3.10.7 Attendance Management Module

The Attendance Module enables teachers to record and monitor student attendance for each course.

Functions

- Record attendance
- Update attendance records
- View attendance history
- Generate attendance reports
- Monitor absenteeism

- Export attendance records
-

3.10.8 Grade Management Module

The Grade Management Module manages student assessment results and overall academic performance.

Functions

- Record grades
 - Update grades
 - Calculate final scores
 - Generate grade reports
 - View student performance
 - Export academic results
-

3.10.9 Announcement and Notification Module

This module facilitates communication between administrators, teachers, and students by distributing academic announcements and notifications.

Functions

- Publish announcements
- Send notifications

- Display important notices
 - Notify assignment deadlines
 - Notify quiz schedules
 - Notify grade publication
 - Broadcast institutional messages
-

3.10.10 Reporting Module

The Reporting Module provides administrators and teachers with analytical reports that support academic planning and decision-making.

Functions

- Generate student reports
 - Generate course reports
 - Generate attendance reports
 - Generate grade reports
 - Produce enrolment statistics
 - Export reports in printable formats
 - Monitor overall academic performance
-

3.10.11 Audit and Activity Logging Module

The Audit Module records significant user activities performed within the system. These records improve accountability, troubleshooting, and system security.

Functions

- Record login activities
 - Record user operations
 - Monitor system access
 - Record administrative actions
 - Track failed login attempts
 - Generate audit reports
-

3.10.12 API Integration Module

The API Integration Module serves as the communication layer between the frontend application and the backend services. It processes requests, validates data, and returns JSON responses.

Functions

- Process RESTful API requests
- Validate incoming requests
- Return JSON responses
- Handle HTTP methods (GET, POST, PUT, DELETE)

- Communicate with the database
 - Handle API errors
 - Manage data exchange between frontend and backend
-

3.10.13 Database Management Module

This module manages all database operations performed by the system and ensures that academic information is stored securely and consistently.

Functions

- Store system data
 - Retrieve records
 - Update information
 - Delete records
 - Manage database relationships
 - Maintain referential integrity
 - Perform database backup and recovery
 - Optimise database performance
-

Summary

The proposed School Learning Management System is organised into specialised modules that collectively support academic administration, teaching, and learning. The Authentication, User Management, Course Management, Learning Materials, Assignment, Quiz, Attendance, Grade, Announcement, Reporting, Audit, API Integration, and Database Management modules work together to provide a secure, scalable, and efficient platform. This modular architecture simplifies system maintenance, enhances scalability, promotes code reusability, and allows future expansion without significantly affecting existing functionality.

3.11 Data Dictionary

The data dictionary provides a description of the major data elements used in the proposed School Learning Management System (LMS). It defines the attributes, data types, and purposes of important fields within the database. The data dictionary serves as a reference for developers, database administrators, and system users by promoting consistency in database design and implementation.

3.11.1 Users Table

Field	Data Type	Description
user_id	INT	Unique identifier for each user
role_id	INT	References the user's role
department_id	INT	References the department
username	VARCHAR(50)	Username for login
email	VARCHAR(100)	User email address
password	VARCHAR(255)	Encrypted password
first_name	VARCHAR(100)	User's first name
last_name	VARCHAR(100)	User's last name
phone	VARCHAR(20)	Contact number
status	ENUM	Account status (Active/Inactive)
created_at	TIMESTAMP	Date account was created

3.11.2 Roles Table

Field	Data Type	Description
role_id	INT	Unique role identifier

Field	Data Type	Description
role_name	VARCHAR(50)	Name of the role
description	TEXT	Role description

3.11.3 Departments Table

Field	Data Type	Description
department_id	INT	Unique department identifier
department_name	VARCHAR(100)	Department name
department_code	VARCHAR(20)	Department code
description	TEXT	Department description

3.11.4 Courses Table

Field	Data Type	Description
course_id	INT	Unique course identifier
department_id	INT	Department offering the course
teacher_id	INT	Assigned teacher
course_code	VARCHAR(20)	Course code

Field	Data Type	Description
course_name	VARCHAR(150)	Course title
description	TEXT	Course description
credit_hours	INT	Credit hours
semester	VARCHAR(20)	Academic semester
academic_year	VARCHAR(20)	Academic year
status	ENUM	Course status

3.11.5 Student Enrolments Table

Field	Data Type	Description
enrolment_id	INT	Enrolment identifier
student_id	INT	Registered student
course_id	INT	Enrolled course
enrolment_date	DATE	Date of enrolment
status	ENUM	Enrolment status

3.11.6 Learning Materials Table

Field	Data Type	Description
material_id	INT	Material identifier
course_id	INT	Associated course
teacher_id	INT	Uploading teacher
title	VARCHAR(200)	Material title
material_type	VARCHAR(50)	Material category
file_path	VARCHAR(255)	Storage location
file_size	BIGINT	File size
upload_date	DATETIME	Upload date
is_archived	BOOLEAN	Archive status

3.11.7 Assignments Table

Field	Data Type	Description
assignment_id	INT	Assignment identifier
course_id	INT	Related course
teacher_id	INT	Assignment creator

Field	Data Type	Description
title	VARCHAR(200)	Assignment title
description	TEXT	Assignment details
due_date	DATETIME	Submission deadline
total_marks	DECIMAL(5,2)	Maximum marks
created_at	TIMESTAMP	Date created

3.11.8 Assignment Submissions Table

Field	Data Type	Description
submission_id	INT	Submission identifier
assignment_id	INT	Related assignment
student_id	INT	Student submitting
file_path	VARCHAR(255)	Submitted file
submission_date	DATETIME	Submission date
marks	DECIMAL(5,2)	Marks awarded
feedback	TEXT	Teacher comments

Field	Data Type	Description
status	ENUM	Submission status

3.11.9 Quizzes Table

Field	Data Type	Description
quiz_id	INT	Quiz identifier
course_id	INT	Associated course
teacher_id	INT	Quiz creator
title	VARCHAR(200)	Quiz title
duration	INT	Time limit (minutes)
total_marks	DECIMAL(5,2)	Total marks
availability_date	DATETIME	Quiz opening date

3.11.10 Grades Table

Field	Data Type	Description
grade_id	INT	Grade identifier
student_id	INT	Student receiving grade

Field	Data Type	Description
course_id	INT	Course
assessment_type	VARCHAR(50)	Assignment, Quiz, Exam
score	DECIMAL(5,2)	Marks obtained
grade	VARCHAR(5)	Letter grade
remarks	VARCHAR(100)	Performance remarks

3.11.11 Attendance Table

Field	Data Type	Description
attendance_id	INT	Attendance identifier
student_id	INT	Student
course_id	INT	Course
attendance_date	DATE	Attendance date
status	ENUM	Present, Absent, Late
remarks	VARCHAR(100)	Attendance remarks

3.11.12 Announcements Table

Field	Data Type	Description
announcement_id	INT	Announcement identifier
user_id	INT	Publisher
title	VARCHAR(200)	Announcement title
content	TEXT	Announcement details
publish_date	DATETIME	Publication date
expiry_date	DATETIME	Expiry date

3.11.13 Audit Logs Table

Field	Data Type	Description
log_id	INT	Log identifier
user_id	INT	User performing activity
action	VARCHAR(150)	Performed action
module	VARCHAR(100)	System module
ip_address	VARCHAR(50)	Client IP address
created_at	DATETIME	Activity timestamp

Summary

The data dictionary defines the major entities and attributes used in the proposed School Learning Management System database. It establishes a common reference for developers and database administrators, ensuring consistency in naming conventions, data types, and relationships. The documented fields support secure storage, efficient retrieval, and accurate management of academic and administrative information while providing a foundation for system implementation and future maintenance.

3.12 Decision Table

The proposed School Learning Management System implements Role-Based Access Control (RBAC) to ensure that users can only perform functions authorised for their respective roles. The decision table below summarises the permissions assigned to Administrators, Teachers, and Students.

Table 3.14: Decision Table for User Roles and Permissions

System Function	Administrator	Teacher	Student
Login	✓	✓	✓
Logout	✓	✓	✓

System Function	Administrator	Teacher	Student
View Dashboard	✓	✓	✓
Manage User Accounts	✓	✗	✗
Manage Roles & Permissions	✓	✗	✗
Manage Departments	✓	✗	✗
Create Courses	✓	✓*	✗
Edit Courses	✓	✓*	✗
Delete Courses	✓	✗	✗
View Courses	✓	✓	✓
Enrol Students	✓	✗	✓**
Upload Learning Materials	✗	✓	✗
Download Learning Materials	✓	✓	✓
Archive Learning Materials	✗	✓	✗
Create Assignments	✗	✓	✗

System Function	Administrator	Teacher	Student
Submit Assignments	X	X	✓
Grade Assignments	X	✓	X
Create Quizzes	X	✓	X
Attempt Quizzes	X	X	✓
Record Attendance	X	✓	X
View Attendance	✓	✓	✓
Publish Announcements	✓	✓	X
View Announcements	✓	✓	✓
View Grades	✓	✓	✓***
Generate Reports	✓	✓****	X
View Audit Logs	✓	X	X
Backup Database	✓	X	X
Manage System Settings	✓	X	X

Key

- $\checkmark$ = Permission Granted
 - $\times$ = Permission Denied
 - $\checkmark^*$ = Teachers can create and manage only the courses assigned to them.
 - $\checkmark^{**}$ = Teachers can generate reports only for their own courses.
 - $\checkmark$ = Students can enrol only in courses made available to them (subject to institutional policy).
 - $\checkmark^*$ = Students can view only their own grades and academic records.
-

Explanation

The decision table illustrates how the proposed Learning Management System enforces access control through Role-Based Access Control (RBAC). Administrators possess full privileges to manage users, departments, courses, reports, audit logs, and system configuration. Teachers are responsible for instructional activities such as managing assigned courses, uploading learning materials, creating assignments and quizzes, recording attendance, grading student work, and publishing announcements. Students have limited access, allowing them to view course information, access learning materials, submit assignments, participate in quizzes, monitor attendance, and view their individual academic results.

By restricting users to only the functions necessary for their responsibilities, the system enhances security, protects sensitive academic information, and maintains the integrity of institutional data.

3.13 Use Case Description

The Use Case Description explains the major interactions between the users of the proposed School Learning Management System (LMS) and the system itself. Each use case represents a specific function performed by an actor to achieve a particular objective. The system has three primary actors: **Administrator**, **Teacher**, and **Student**, each interacting with the system according to their assigned roles and privileges.

Table 3.15: Use Case Description

Use Case	Primary Actor	Description
Login	Administrator, Teacher, Student	Authenticates users before granting access to the system.
Logout	Administrator, Teacher, Student	Ends the current user session securely.
View Dashboard	Administrator, Teacher, Student	Displays role-specific information and system activities.

Use Case	Primary Actor	Description
Manage Users	Administrator	Creates, updates, deactivates, and manages user accounts and roles.
Manage Departments	Administrator	Creates, updates, and manages academic departments.
Create Course	Administrator, Teacher	Creates a new course and assigns course information.
Update Course	Administrator, Teacher	Modifies course details and settings.
View Courses	Administrator, Teacher, Student	Displays available courses according to user privileges.
Enrol Student	Administrator	Registers students for selected courses.
Upload Learning Material	Teacher	Uploads lecture notes, presentations, videos, and other learning resources.
Download Learning Material	Teacher, Student	Downloads learning materials from enrolled courses.
Archive Learning Material	Teacher	Archives outdated or inactive learning materials.

Use Case	Primary Actor	Description
Create Assignment	Teacher	Creates assignments for students enrolled in a course.
Submit Assignment	Student	Uploads completed assignments before the submission deadline.
Grade Assignment	Teacher	Evaluates submitted assignments and records marks.
Create Quiz	Teacher	Creates online quizzes for assessment.
Attempt Quiz	Student	Completes quizzes within the allocated time.
Record Attendance	Teacher	Records students' attendance during lectures.
View Attendance	Teacher, Student	Displays attendance records for courses.
Record Grades	Teacher	Records examination and assessment scores.
View Grades	Student, Teacher	Displays student academic performance.
Publish Announcement	Administrator, Teacher	Posts announcements and important academic information.
View Announcements	Administrator, Teacher, Student	Displays institutional and course announcements.

Use Case	Primary Actor	Description
Generate Reports	Administrator, Teacher	Produces reports on academic activities, attendance, grades, and users.
View Audit Logs	Administrator	Reviews user activities and system events for security and monitoring.
Backup Database	Administrator	Performs database backup to prevent data loss.

Description of Major Use Cases

Login

Actor: Administrator, Teacher, Student

Purpose:

To authenticate users and grant secure access to the Learning Management System.

Precondition:

- User account exists.
- Valid username and password are available.

Main Flow:

1. User enters login credentials.
2. System validates the credentials.

3. User is authenticated.
4. Appropriate dashboard is displayed.

Postcondition:

- User session is created.
 - Access permissions are assigned according to the user's role.
-

Upload Learning Material

Actor: Teacher

Purpose:

To provide students with lecture notes, presentations, videos, and other educational resources.

Precondition:

- Teacher is logged into the system.
- Teacher is assigned to the selected course.

Main Flow:

1. Teacher selects a course.
2. Teacher enters material details.
3. Teacher uploads the file.
4. System validates the file.

5. Material is stored in the database and file storage.
6. Students can access the uploaded material.

Postcondition:

- Learning material becomes available to authorised students.
-

Submit Assignment

Actor: Student

Purpose:

To submit completed coursework electronically.

Precondition:

- Student is enrolled in the course.
- Assignment is available and submission deadline has not expired.

Main Flow:

1. Student selects an assignment.
2. Student uploads the completed work.
3. System validates the uploaded file.
4. Submission is stored successfully.
5. Confirmation message is displayed.

Postcondition:

- Assignment is recorded and becomes available for grading.
-

Grade Assignment

Actor: Teacher

Purpose:

To evaluate submitted assignments and provide marks and feedback.

Precondition:

- Student submissions are available.
- Teacher has permission to access the course.

Main Flow:

1. Teacher views submitted assignments.
2. Teacher reviews the submission.
3. Teacher enters marks and feedback.
4. System saves the assessment.

Postcondition:

- Student grades are updated and become available for viewing.
-

Generate Reports

Actor: Administrator, Teacher

Purpose:

To produce academic and administrative reports for decision-making.

Precondition:

- User is authenticated.
- User has permission to generate reports.

Main Flow:

1. User selects report type.
2. User specifies report parameters.
3. System retrieves relevant data.
4. Report is generated.
5. User views or downloads the report.

Postcondition:

- Report is available for printing or export.

Summary

The use case descriptions complement the Use Case Diagram by providing detailed explanations of how users interact with the proposed School Learning Management System. They define the actors involved, the objectives of each interaction, the conditions required before execution, the sequence of activities performed, and the expected

outcomes. Together, the Use Case Diagram and Use Case Descriptions provide a comprehensive specification of the system's functional behaviour and support the implementation of a secure, role-based, and user-centred Learning Management System.

3.14 Data Flow Diagram (DFD) Explanation

The Data Flow Diagram (DFD) illustrates how data moves through the proposed School Learning Management System (LMS). It identifies the major processes, data stores, external entities, and the flow of information among them. The DFD demonstrates how users interact with the system and how information is processed, stored, and retrieved.

The major processes of the proposed system are explained below.

3.14.1 Process 1: User Authentication

The User Authentication process verifies the identity of users before granting access to the Learning Management System. All users, including administrators, teachers, and students, must successfully authenticate before accessing protected resources.

Input

- Username
- Password

Processing

- Validate user credentials.
- Compare encrypted password with the stored password.
- Verify account status.
- Determine user role.
- Create user session.
- Record login activity.

Output

- Successful login confirmation.
- User dashboard.
- Error message for invalid credentials.
- User session information.

Data Store

- Users Table
- Roles Table
- Audit Logs Table

3.14.2 Process 2: Course Management

The Course Management process allows authorised users to create, update, manage, and retrieve course information.

Input

- Course code
- Course title
- Course description
- Teacher assignment
- Department information
- Academic year
- Semester

Processing

- Validate course information.
- Create new course.
- Update existing course.
- Assign teachers.
- Save course information.

Output

- Course successfully created.
- Updated course details.
- List of available courses.

Data Store

- Courses Table
 - Departments Table
 - Users Table
-

3.14.3 Process 3: Learning Material Management

This process enables teachers to upload, organise, archive, and manage digital learning resources.

Input

- Course selection
- Material title
- Material type
- Uploaded file

Processing

- Validate uploaded file.
- Store file on the server.
- Save metadata.
- Associate material with the selected course.
- Archive materials when necessary.

Output

- Upload confirmation.
- Updated material list.
- Downloadable learning resources.

Data Store

- Learning Materials Table
 - Courses Table
-

3.14.4 Process 4: Assignment Management

The Assignment Management process supports assignment creation, submission, grading, and feedback.

Input

Teacher

- Assignment details
- Due date
- Instructions

Student

- Assignment file
- Submission comments

Processing

- Create assignments.
- Validate submissions.
- Store uploaded files.
- Grade assignments.
- Record feedback.

Output

- Assignment created.
- Submission confirmation.
- Student grades.
- Teacher feedback.

Data Store

- Assignments Table
- Assignment Submissions Table
- Grades Table

3.14.5 Process 5: Quiz Management

This process manages online quizzes used for continuous assessment.

Input

- Quiz information

- Questions
- Student answers

Processing

- Create quizzes.
- Display questions.
- Record answers.
- Automatically calculate objective scores.
- Store quiz results.

Output

- Quiz results.
- Student scores.
- Performance reports.

Data Store

- Quizzes Table
- Quiz Attempts Table
- Grades Table

3.14.6 Process 6: Attendance Management

The Attendance Management process records and maintains student attendance for each course.

Input

- Student list
- Attendance status
- Attendance date

Processing

- Validate attendance records.
- Save attendance information.
- Update attendance history.

Output

- Attendance confirmation.
- Attendance reports.
- Attendance statistics.

Data Store

- Attendance Table
 - Courses Table
 - Users Table
-

3.14.7 Process 7: Grade Management

The Grade Management process records and manages students' academic performance.

Input

- Student marks
- Assessment type
- Course information

Processing

- Validate grades.
- Calculate totals.
- Compute final grades.
- Store academic results.

Output

- Student grades.
- Academic transcript.
- Grade reports.

Data Store

- Grades Table
- Students Table
- Courses Table

3.14.8 Process 8: Announcement Management

This process enables administrators and teachers to communicate important academic information to users.

Input

- Announcement title
- Announcement content
- Publication date
- Expiry date

Processing

- Validate announcement.
- Store announcement.
- Publish to relevant users.

Output

- Published announcement.
- User notifications.

Data Store

- Announcements Table
-

3.14.9 Process 9: Report Generation

The Report Generation process provides academic and administrative reports that support institutional decision-making.

Input

- Report type
- Search criteria
- Date range

Processing

- Retrieve relevant data.
- Analyse records.
- Generate reports.
- Format report output.

Output

- Student reports.
- Attendance reports.
- Grade reports.
- Course reports.
- Statistical summaries.

Data Store

- Users Table
 - Courses Table
 - Attendance Table
 - Grades Table
 - Assignments Table
 - Learning Materials Table
-

3.14.10 Process 10: Audit Logging

The Audit Logging process records significant activities performed within the system to enhance accountability and security.

Input

- User activity
- Login information
- System events

Processing

- Record user action.
- Store timestamp.
- Save IP address.
- Save activity description.

Output

- Audit trail.
- Security reports.
- User activity history.

Data Store

- Audit Logs Table
-

Summary

The Data Flow Diagram illustrates the movement of information throughout the proposed School Learning Management System. Each process transforms user inputs into meaningful outputs while interacting with the appropriate database tables. Authentication controls access to the system, course and learning material management support teaching and learning activities, assignment, quiz, attendance, and grade management facilitate academic evaluation, while announcements, reporting, and audit logging improve communication, administration, and accountability. Together, these processes ensure efficient, secure, and reliable management of academic information within the Learning Management System.

3.15 Technology Selection

The successful implementation of the proposed School Learning Management System (LMS) depends on selecting appropriate technologies that provide reliability, scalability, maintainability, and cost-effectiveness. The technologies used were selected based on their compatibility, performance, ease of development, community support, and suitability for web-based educational applications.

3.15.1 HTML5

HyperText Markup Language version 5 (HTML5) was selected as the primary markup language for developing the frontend interface of the proposed Learning Management System.

HTML5 was chosen because it:

- Provides a standard structure for web applications.
- Supports semantic elements that improve accessibility.
- Is compatible with all modern web browsers.
- Supports multimedia content without requiring additional plugins.
- Integrates easily with CSS3 and JavaScript.

HTML5 enables the development of responsive and well-structured web pages that are accessible across different devices.

3.15.2 CSS3

Cascading Style Sheets version 3 (CSS3) was selected to design and style the graphical user interface of the system.

CSS3 provides several advantages, including:

- Responsive web design.
- Flexible layouts using Flexbox and CSS Grid.
- Attractive user interfaces.
- Animation and transition effects.
- Improved user experience across desktop and mobile devices.

CSS3 separates presentation from application logic, making the system easier to maintain and update.

3.15.3 JavaScript

JavaScript was selected to provide interactivity and dynamic functionality within the frontend of the application.

JavaScript enables the system to:

- Validate user input before submission.
- Communicate with backend APIs asynchronously.
- Update page content without full-page reloads.

- Display notifications and alerts.
- Improve responsiveness and user experience.

Its widespread browser support and integration with modern web technologies make it suitable for developing interactive educational applications.

3.15.4 PHP 8

PHP version 8 was selected as the backend programming language responsible for implementing the business logic and RESTful APIs of the Learning Management System.

PHP 8 was chosen because it:

- Integrates efficiently with MySQL databases.
- Supports object-oriented programming.
- Provides high execution performance.
- Includes improved security features.
- Supports modern programming practices.
- Has extensive documentation and community support.

PHP processes client requests, manages authentication, performs database operations, and returns JSON responses to the frontend.

3.15.5 MySQL

MySQL was selected as the relational database management system for storing and managing academic and administrative data.

The reasons for selecting MySQL include:

- High performance.
- Reliability.
- Open-source licensing.
- Strong security features.
- Support for relational database design.
- Compatibility with PHP.
- Efficient data indexing and query optimisation.

MySQL provides secure storage for users, courses, assignments, learning materials, grades, attendance records, and other institutional data.

3.15.6 REST API

The proposed Learning Management System adopts a **Representational State Transfer (REST)** architecture for communication between the frontend and backend.

REST APIs were selected because they:

- Separate the frontend from the backend.
- Improve scalability.

- Support multiple client applications.
- Simplify integration with mobile and desktop applications.
- Use standard HTTP protocols.
- Facilitate modular software development.

RESTful architecture allows the frontend to communicate with backend services through well-defined API endpoints.

3.15.7 JSON

JavaScript Object Notation (JSON) was selected as the standard format for exchanging information between the frontend and backend.

JSON was chosen because it:

- Is lightweight.
- Is easy to read and write.
- Is language independent.
- Reduces network bandwidth usage.
- Is supported by almost every programming language.

All API responses in the proposed system are transmitted in JSON format, enabling efficient communication between application components.

3.15.8 Fetch API

The **Fetch API** was selected as the mechanism for sending asynchronous HTTP requests from the frontend to the backend REST APIs.

Its advantages include:

- Simpler syntax than traditional AJAX.
- Native browser support.
- Promise-based asynchronous programming.
- Improved readability.
- Efficient handling of JSON data.
- Better error handling.

The Fetch API enables users to perform system operations without reloading web pages, resulting in a smoother user experience.

3.15.9 XAMPP

XAMPP was selected as the local development environment for implementing and testing the proposed Learning Management System.

XAMPP provides:

- Apache Web Server.
- PHP Runtime Environment.

- MySQL Database Server.
- phpMyAdmin Database Administration Tool.

The development environment simplifies installation, configuration, and testing by integrating all required software into a single package.

3.15.10 Visual Studio Code (VS Code)

Visual Studio Code was selected as the primary Integrated Development Environment (IDE) for developing the proposed system.

The reasons for selecting VS Code include:

- Lightweight and high-performance editor.
- Excellent support for PHP, HTML, CSS, and JavaScript.
- Integrated Git and GitHub support.
- Intelligent code completion and debugging.
- Extensive extension marketplace.
- Cross-platform compatibility.

VS Code improved developer productivity through features such as syntax highlighting, code navigation, integrated terminal, and version control integration.

3.15.11 Git and GitHub

Git was used as the version control system, while **GitHub** served as the remote repository for source code management and collaboration.

The combination of Git and GitHub provides:

- Version tracking.
- Source code backup.
- Collaboration among developers.
- Branch management.
- Change history.
- Easy deployment and recovery.

These tools helped ensure effective project management throughout the software development process.

Summary

The technologies selected for the proposed School Learning Management System were chosen based on their reliability, compatibility, scalability, security, and ease of development. HTML5, CSS3, and JavaScript provide a responsive and interactive user interface, while PHP 8 and MySQL deliver a robust backend capable of securely managing academic data. RESTful APIs, JSON, and the Fetch API enable efficient communication between the frontend and backend, supporting the project's API-first

architecture. XAMPP provides a convenient local development environment, whereas Visual Studio Code, Git, and GitHub facilitate efficient software development and version control. Collectively, these technologies provide a modern, secure, and scalable platform capable of meeting the functional and non-functional requirements of the proposed Learning Management System.

3.16 Security Considerations

Security is a critical aspect of the proposed School Learning Management System (LMS) because the system stores sensitive academic and personal information, including user credentials, grades, attendance records, assignments, and learning materials. The system incorporates multiple security mechanisms to protect data confidentiality, integrity, and availability while preventing unauthorized access and common web-based attacks.

The major security measures implemented in the proposed system are described below.

3.16.1 Password Hashing

Passwords are not stored in plain text within the database. Instead, the system uses the **bcrypt** hashing algorithm provided by PHP's `password_hash()` function to securely encrypt user passwords before storage.

During login, the entered password is verified using the `password_verify()` function rather than comparing plain-text passwords.

The use of `bcrypt` provides the following benefits:

- Prevents exposure of user passwords if the database is compromised.
 - Uses one-way encryption that cannot easily be reversed.
 - Includes automatic salting to strengthen password security.
 - Protects against brute-force and rainbow table attacks.
-

3.16.2 User Authentication

Authentication ensures that only legitimate users gain access to the system.

The authentication process involves:

1. User enters a username or email address and password.
2. The system validates the submitted credentials.
3. The password hash is verified.
4. User identity is confirmed.
5. User role is retrieved.
6. Access is granted based on assigned permissions.

If authentication fails, an appropriate error message is returned without revealing sensitive system information.

Only authenticated users are permitted to access protected API endpoints.

3.16.3 Session Management

The proposed LMS uses secure PHP sessions to maintain authenticated user sessions after successful login.

The session management mechanism includes:

- Automatic session creation after login.
- Session regeneration after successful authentication to prevent session fixation attacks.
- Automatic session timeout after a period of inactivity.
- Session destruction during logout.
- Validation of active sessions before processing protected requests.
- Storage of user identity and role within the session.

Proper session management prevents unauthorized users from accessing protected resources after a session has expired.

3.16.4 Input Validation and Sanitisation

All user input is validated before being processed by the application.

Validation includes:

- Required field verification.
- Data type validation.
- Email format validation.
- File type validation.
- File size validation.
- Length restrictions.
- Numeric range validation.

Input sanitisation removes malicious characters before storing or displaying data.

These validation mechanisms help prevent:

- Invalid data entry.
- Cross-Site Scripting (XSS).
- Malicious file uploads.
- Unexpected application errors.

3.16.5 Prepared Statements

The proposed system uses **prepared SQL statements** with parameter binding for all database operations.

Prepared statements separate SQL commands from user-supplied data, preventing malicious SQL code from being executed.

Example database operations include:

- User authentication.
- Course creation.
- Assignment submission.
- Grade recording.
- Attendance management.

Prepared statements significantly reduce the risk of **SQL Injection attacks**, which are among the most common web application vulnerabilities.

3.16.6 Role-Based Access Control (RBAC)

The proposed Learning Management System implements **Role-Based Access Control (RBAC)** to regulate user permissions.

Three primary user roles are defined:

- Administrator
- Teacher
- Student

Each role is granted only the permissions required to perform its assigned responsibilities.

Examples include:

Administrator

- Manage users
- Manage departments
- Manage courses
- View reports
- Manage system settings
- Backup database
- View audit logs

Teacher

- Upload learning materials
- Create assignments
- Create quizzes
- Record attendance
- Grade students
- Publish announcements

Student

- View enrolled courses
- Download learning materials
- Submit assignments

- Attempt quizzes
- View grades
- View attendance

RBAC enforces the principle of **least privilege**, ensuring that users cannot access functions beyond their authorised responsibilities.

3.16.7 File Upload Security

The Learning Management System allows teachers and students to upload learning materials and assignment submissions. To prevent malicious uploads, the following controls are implemented:

- Validation of permitted file types (e.g., PDF, DOCX, PPTX).
- File size restrictions.
- Renaming uploaded files with unique identifiers.
- Secure storage outside publicly writable directories where applicable.
- Scanning and validation before database registration.

These measures reduce the risk of malicious file execution and unauthorized file access.

3.16.8 API Security

Since the proposed system adopts an API-first architecture, backend services are protected through several security mechanisms.

These include:

- Authentication before accessing protected endpoints.
- Session verification for each API request.
- Role-based authorization checks.
- Validation of request parameters.
- Standard HTTP status codes for error handling.
- JSON-only responses to ensure consistent communication.

These controls ensure that API resources are accessed only by authorised users.

3.16.9 Error Logging and Audit Trails

The system maintains comprehensive logs of significant user and system activities.

Examples of logged events include:

- User login and logout.
- Failed authentication attempts.
- Course creation and updates.
- Assignment submissions.
- Grade modifications.

- Administrative actions.
- System errors.

Audit logs support troubleshooting, monitoring, and accountability while providing evidence of user activities.

3.16.10 Database Backup and Recovery

To minimise the risk of data loss, the proposed LMS incorporates database backup and recovery mechanisms.

These include:

- Scheduled database backups.
- Secure backup storage.
- Recovery procedures for restoring lost or corrupted data.
- Protection of backup files through restricted access.

Regular backups ensure business continuity in the event of hardware failure, software malfunction, or accidental data deletion.

Summary

Security is a fundamental consideration in the design and implementation of the proposed School Learning Management System. The system employs multiple layers of protection,

including **bcrypt password hashing, secure authentication, session management, input validation and sanitisation, prepared SQL statements, and Role-Based Access Control (RBAC)**. Additional measures such as secure file upload handling, API protection, audit logging, and regular database backups further strengthen the system against common security threats. Collectively, these mechanisms ensure the confidentiality, integrity, and availability of academic information while providing a secure and reliable learning environment for administrators, teachers, and students.

3.17 Constraints and Assumptions

The successful implementation and operation of the proposed School Learning Management System (LMS) depend on certain assumptions regarding the operating environment and user behaviour. Additionally, the system is subject to technical and operational constraints that may affect its performance and deployment. Understanding these assumptions and constraints provides a realistic context for evaluating the capabilities and limitations of the proposed solution.

3.17.1 Assumptions

The development and deployment of the proposed Learning Management System are based on the following assumptions:

- Users have access to a computer, laptop, tablet, or smartphone capable of accessing the system.
- Users have a stable internet connection when accessing the web application remotely.
- Each user possesses valid login credentials issued by the system administrator.
- Administrators are responsible for creating and maintaining user accounts and assigning appropriate roles.
- Teachers will upload learning materials, assignments, quizzes, and attendance records regularly.
- Students will use the system responsibly to access learning materials, submit assignments, and participate in assessments.
- The institution possesses the necessary server infrastructure to host the application and database.
- The MySQL database server will remain operational and accessible whenever the application is in use.
- Users possess basic computer literacy required to navigate the Learning Management System.
- Appropriate security policies and backup procedures are implemented by the institution.

3.17.2 Constraints

Despite the advantages of the proposed system, several limitations may affect its implementation and operation.

Internet Connectivity

Since the Learning Management System is web-based, users require reliable internet connectivity to access system resources. Poor or unstable internet connections may delay login, file uploads, assignment submissions, and access to learning materials.

Browser Compatibility

The system is designed to function on modern web browsers such as Google Chrome, Microsoft Edge, Mozilla Firefox, and Safari. Older or unsupported browsers may not correctly display some interface components or support all system features.

Server Availability

The availability of the system depends on the operational status of the web server and database server. Hardware failures, maintenance activities, or server downtime may temporarily prevent users from accessing the system.

Hardware Limitations

System performance may be affected by the processing capability, memory capacity, and storage resources of both the server and client devices. Institutions with limited hardware resources may experience reduced performance under heavy workloads.

Scalability

Although the system is designed to accommodate institutional growth, extremely large numbers of simultaneous users may require additional server resources or infrastructure upgrades to maintain acceptable performance.

Power Supply

Reliable electrical power is essential for both the server and client devices. Power outages may interrupt access to the system and affect ongoing academic activities, particularly in environments without backup power solutions.

Data Security Risks

Although comprehensive security mechanisms have been implemented, no web application can eliminate cybersecurity risks. Threats such as phishing, malware, weak user passwords, and social engineering attacks may still compromise user accounts if appropriate security practices are not followed.

File Storage Capacity

The storage of learning materials, assignment submissions, and multimedia resources requires sufficient disk space. As the volume of uploaded content increases over time, additional storage capacity may become necessary.

Dependence on Third-Party Software

The operation of the proposed system depends on external software components such as Apache, PHP, MySQL, XAMPP, and modern web browsers. Changes, updates, or compatibility issues in these technologies may require corresponding updates to the application.

3.17.3 Mitigation Measures

To minimise the impact of the identified constraints, the following measures are recommended:

- Deploy the system on reliable server infrastructure with adequate processing power and storage.
- Perform regular database backups and system maintenance.
- Encourage users to access the system using supported web browsers.
- Implement strong password policies and user security awareness programmes.
- Monitor server performance and upgrade infrastructure as user numbers increase.

- Provide uninterrupted power supply (UPS) and backup power solutions for the server environment.
 - Ensure reliable internet connectivity for both administrators and end users.
 - Regularly update software components to address security vulnerabilities and improve compatibility.
-

Summary

The proposed School Learning Management System is developed based on several operational assumptions regarding user access, institutional infrastructure, and system administration. At the same time, its implementation may be influenced by constraints such as internet connectivity, browser compatibility, server availability, hardware resources, storage capacity, and cybersecurity risks. By recognising these assumptions and constraints and implementing appropriate mitigation measures, the institution can improve the reliability, security, and long-term sustainability of the proposed Learning Management System.

3.18 Chapter Summary

This chapter presented the methodology and system design adopted for the development of the proposed School Learning Management System (LMS). It began by analysing the

existing manual and partially digitised academic management processes, identifying their limitations in areas such as user management, course administration, assignment handling, communication, reporting, and security. Based on these findings, the proposed system workflow was introduced to demonstrate how the new LMS addresses the identified challenges through a web-based, API-first architecture.

The chapter further described the functional and non-functional requirements of the system, together with the hardware, software, and database requirements necessary for successful implementation. The overall system architecture, including the RESTful API architecture, was presented to illustrate the interaction between the frontend, backend, authentication layer, and MySQL database. In addition, the major system modules were described, highlighting their respective responsibilities in supporting academic and administrative activities.

The chapter also documented the database design through a data dictionary, explained the role-based decision table, and provided detailed descriptions of the system use cases and Data Flow Diagram (DFD) processes. Furthermore, the technologies selected for system development, including HTML5, CSS3, JavaScript, PHP 8, MySQL, REST APIs, JSON, the Fetch API, and XAMPP, were justified based on their suitability, compatibility, and performance.

Finally, the chapter discussed the security mechanisms incorporated into the system, including password hashing, authentication, session management, input validation, prepared SQL statements, and Role-Based Access Control (RBAC). The assumptions and constraints associated with the proposed system were also presented to provide a realistic understanding of the operational environment and potential limitations.

Overall, this chapter established the technical foundation for implementing the proposed School Learning Management System. The design decisions and methodologies discussed provide a secure, scalable, maintainable, and user-centred solution capable of improving academic administration, enhancing teaching and learning activities, and supporting efficient educational service delivery. The next chapter presents the implementation, testing, and evaluation of the developed system.